SOLID Principles

Five Rules That Make Code Easier to Change

Advanced Object-Oriented Programming | Lecture 9

Quick Recap from Last Lecture

- Working code ≠ Good code — it must also be readable, maintainable, and extensible
- Code smells like rigidity, fragility, and duplication are warning signs
- Loose coupling (interfaces) > Tight coupling (concrete classes)
- High cohesion — each class should do ONE thing well
- Composition (HAS-A) > Inheritance (IS-A) in most cases

Today: SOLID gives us concrete rules to achieve all of the above.

What is SOLID?

S	**Single Responsibility**	*One class, one job*
O	**Open/Closed**	*Extend, don't modify*
L	**Liskov Substitution**	*Children honor parent contracts*
I	**Interface Segregation**	*Keep interfaces small*
D	**Dependency Inversion**	*Depend on abstractions*

S — Single Responsibility Principle

A class should have only ONE reason to change

Translation: Each class should do ONE thing well

❌ **Violates SRP — 3 reasons to change!**

```
class Student {
  String name;
  int rollNo;

  double calculateGPA() {
    // academic logic
  }
  void saveToDatabase() {
    // persistence logic
  }
  String generateMarksheet() {
    // formatting logic
  }
}
```

✅ **Follows SRP — each class, one job**

```
class Student {
  String name;
  int rollNo;
}

class GPACalculator {
  double calculate(Student s) { }
}

class StudentRepository {
  void save(Student s) { }
}

class MarksheetGenerator {
  String generate(Student s) { }
}
```

💡 **GPA formula changes? Only edit GPACalculator. Switch DB? Only edit StudentRepository. Neither affects the other!**

O — Open/Closed Principle

Open for extension, closed for modification

Translation: Add new features WITHOUT changing existing, tested code

❌ **Violates OCP — must edit for every new type**

```
class FeeCalculator {
  double calculate(String studentType) {
    if (studentType.equals("regular")) {
      return 50000;
    } else if (studentType.equals("scholarship")) {
      return 10000;
    }
    // New type = must MODIFY this method!
    // What if we add "international"?
    // What if we add "sports_quota"?
    // This method keeps growing...
    return 0;
  }
}
```

✅ **Follows OCP — add new class, don't modify**

```
interface FeePolicy {
  double calculateFee();
}

class RegularFee implements FeePolicy {
  public double calculateFee() {
    return 50000;
  }
}

class ScholarshipFee implements FeePolicy {
  public double calculateFee() {
    return 10000;
  }
}
// New type? Just add a new class!
// class InternationalFee implements FeePolicy
```

Key insight: The if-else chain is the code smell. An interface + polymorphism is the cure.

L — Liskov Substitution Principle

Subclasses must be usable wherever the parent class is expected

Translation: If your code works with a parent type, swapping in any child type must NOT break it

❌ **Violates LSP — Penguin breaks Bird's contract**

```
class Bird {
  void fly() { System.out.println("Flying!"); }  // All birds can fly... right?
}

class Sparrow extends Bird { }  // ✓ Sparrows can fly

class Penguin extends Bird {
  void fly() { throw new UnsupportedOperationException("Can't fly!"); }  // ✗ Breaks the contract!
}

// This code looks correct but CRASHES at runtime:
void makeBirdFly(Bird bird) {
  bird.fly();  // Works for Sparrow, EXPLODES for Penguin!
}
```

⚠ The problem: Bird promises "I can fly," but Penguin breaks that promise. Anyone calling fly() on a Bird can't trust it.

LSP: The Fix — Separate What's Different

```
// Step 1: Bird only promises what ALL birds can do
class Bird {
  void eat() { System.out.println("Eating"); }
}

// Step 2: Separate interface for flying
interface Flyable {
  void fly();
}

// Step 3: Only flying birds implement Flyable
class Sparrow extends Bird implements Flyable {
  public void fly() { System.out.println("Sparrow flying!"); }
}

class Penguin extends Bird {
  // No fly() method — Penguin never promises to fly!
  void swim() { System.out.println("Penguin swimming!"); }
}
```

Now it's safe:

```
void makeFly(Flyable f) { f.fly(); }  // Only accepts things that CAN fly
makeFly(new Sparrow());  // ✓ Works
// makeFly(new Penguin()); // ✗ Won't even compile! Caught at compile time, not runtime.
```

I — Interface Segregation Principle

Don't force classes to implement methods they don't need

Translation: Keep interfaces small and focused

❌ Fat Interface

```
interface SmartDevice {
  void call();
  void browseWeb();
  void takePhoto();
  void makePayment();
}

// BasicPhone is FORCED to implement
// methods it can't support!
class BasicPhone implements SmartDevice {
  public void call() { /* OK */ }
  public void browseWeb() { /* Can't! */ }
  public void takePhoto() { /* No camera! */ }
  public void makePayment() { /* No NFC! */ }
}
```

✅ Segregated Interfaces

```
interface Callable    { void call(); }
interface Browsable   { void browseWeb(); }
interface Camera      { void takePhoto(); }
interface PaymentNFC  { void makePayment(); }

// BasicPhone: only what it can do
class BasicPhone implements Callable {
  public void call() { /* works! */ }
}

// Smartphone: implements all
class Smartphone implements
    Callable, Browsable, Camera, PaymentNFC {
  // implements everything
}
```

Many small interfaces > One giant interface. Each class picks only what it needs.

D — Dependency Inversion Principle

Depend on abstractions (interfaces), not concrete classes

Translation: Don't hardcode "new SpecificClass()" — accept interfaces instead

❌ **Tight coupling — locked to MySQL**

```
class StudentService {
  // Directly creates MySQL — can't switch!
  private MySQLDatabase db =
      new MySQLDatabase();

  void enroll(Student s) {
    db.insert(s);  // Locked to MySQL
  }
}
```

✅ **Depends on abstraction — flexible!**

```
interface Database {
  void insert(Student s);
}

class StudentService {
  private Database db; // interface!

  StudentService(Database db) {
    this.db = db; // injected!
  }

  void enroll(Student s) { db.insert(s); }
}
```

```
// In main: pass in whichever implementation you need
StudentService service = new StudentService(new MySQLDatabase());  // for production
StudentService test    = new StudentService(new FakeDatabase());   // for testing!
```

DIP: Understanding Constructor Injection

This pattern shows up everywhere in professional Java code. Let's break it down step by step.

```java
// STEP 1: Define WHAT we need (the interface)
interface MessageSender {
  void send(String to, String message);
}

// STEP 2: Create different implementations
class EmailSender implements MessageSender {
  public void send(String to, String msg) {
    System.out.println("Emailing " + to + ": " + msg);
  }
}
class SMSSender implements MessageSender {
  public void send(String to, String msg) {
    System.out.println("Texting " + to + ": " + msg);
  }
}

// STEP 3: The service accepts ANY MessageSender via constructor
class AlertService {
  private MessageSender sender;              // holds the interface
  AlertService(MessageSender sender) {       // accepts via constructor
    this.sender = sender;                    // stores it
  }
  void sendAlert(String to, String msg) {
    sender.send(to, msg);                    // uses it
  }
}
```

The caller decides which implementation. The service doesn't care. That's the inversion!

Refactor This Payment Processor

```
class PaymentProcessor {
  void processPayment(String method, double amount) {
    if (method.equals("credit_card")) {
      System.out.println("Charging credit card: Rs " + amount);
      // ... credit card API call
    } else if (method.equals("upi")) {
      System.out.println("Processing UPI: Rs " + amount);
      // ... UPI API call
    } else if (method.equals("net_banking")) {
      System.out.println("Net banking transfer: Rs " + amount);
      // ... net banking API call
    }
    saveTransaction(method, amount);  // Also saves to DB here!
  }
  void saveTransaction(String method, double amount) {
     // Implementation here.
  }
}
```

Questions:

1. Which TWO SOLID principles does this violate? (Hint: SRP + ?)
2. Create a Payment interface with a process(double amount) method
3. Create CreditCardPayment, UPIPayment, NetBankingPayment classes
4. **Where should saveTransaction() live? (Hint: Think SRP)**

Which SOLID Principles Are Violated?

```
interface Animal {
  void walk();
  void swim();
  void fly();
}

class Dog implements Animal {
  public void walk() { System.out.println("Dog walking"); }
  public void swim() { System.out.println("Dog swimming"); }
  public void fly()  { throw new UnsupportedOperationException(); }  // Dogs can't fly!
}

class AnimalTracker {
  private MySQLDatabase db = new MySQLDatabase();
  void track(Dog dog) {  // Takes concrete Dog, not interface
    db.save(dog.toString());
  }
}
```

For each violation, write down:

1. Which SOLID principle is violated? (There are at least 3!)
2. Which specific line(s) of code show the violation?
3. How would you fix it?

Hint: Look for ISP, LSP, and DIP violations

Refactor This Class Using ALL SOLID Principles

```
class LibrarySystem {
  private MySQLDatabase db = new MySQLDatabase();

  void addBook(String title) { db.insert(title); }

  void issueBook(String title, String student) {
    db.update(title, "issued");
    System.out.println("Email: " + student + ", you got " + title);
  }

  double calculateFine(String type, int daysLate) {
    if (type.equals("student"))       return daysLate * 2;
    else if (type.equals("faculty"))  return daysLate * 1;
    else if (type.equals("external")) return daysLate * 5;
    return 0;
  }

  void generateReport() {
    // generates PDF report of all books
  }
}
```

Your Tasks:

1. Identify ALL SOLID violations (there are at least 4)
2. Apply SRP: Split into focused classes (BookRepository, IssueService, FineCalculator, ...)
3. Apply OCP: Make calculateFine extensible without modification
4. Apply DIP: Remove the direct MySQLDatabase dependency
5. **Write a main() method that wires everything together**

SOLID at a Glance

	Principle	The Rule	Code Smell to Watch For
S	**Single Responsibility**	One class, one job	*Class has multiple reasons to change*
O	**Open/Closed**	Extend, don't modify	*if-else chains that grow with every new type*
L	**Liskov Substitution**	Children honor contracts	*Subclass throws UnsupportedOperationException*
I	**Interface Segregation**	Small, focused interfaces	*Class forced to implement irrelevant methods*
D	**Dependency Inversion**	Depend on abstractions	*"new ConcreteClass()" inside another class*

These principles prepare us for Design Patterns — coming next!

What We Learned

1. SRP: Split classes so each has one job and one reason to change
2. OCP: Use interfaces + polymorphism instead of growing if-else chains
3. LSP: Never make a child class break a parent's promises
4. ISP: Many small interfaces beat one big interface
5. DIP: Accept dependencies through constructors, not with 'new'

Next Lecture: Design Patterns — Proven solutions built on everything we've learned →